

PRACTICAL WEEK 11 :

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Code:

```
File: nnn.c

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

// Shared variable
int counter = 0;

// Thread function
void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    // Create multiple threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    // Wait for all threads to finish
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join");
            return 1;
        }
    }

    printf("Expected counter value: %d\n",
          NUM_THREADS * INCREMENTS);

    printf("Actual counter value: %d\n", counter);

    return 0;
}
```

Output:

```
Last login: Thu Sep 24 16:47:49 on tty000
screenikhlreddy@Strees-MacBook-Pro ~ % nano nnn.c

screenikhlreddy@Strees-MacBook-Pro ~ % gcc nnn.c -o nnn
screenikhlreddy@Strees-MacBook-Pro ~ % ./nnn
Expected counter value: 400000
Actual counter value: 128618
screenikhlreddy@Strees-MacBook-Pro ~ %
```

2. Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Code:

```
File: 09.c

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

// Shared variable
int counter = 0;

// Mutex
pthread_mutex_t lock;

// Thread function
void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        // Lock the critical section
        pthread_mutex_lock(&lock);

        // Update shared variable
        counter++;

        // Unlock the critical section
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    // Initialize mutex
    if (pthread_mutex_init(&lock, NULL) != 0)
    {
        perror("Mutex initialization failed\n");
        return 1;
    }

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create\n");
            return 1;
        }
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join\n");
            return 1;
        }
    }

    // Display results
    printf("Expected counter value: %d\n",
          NUM_THREADS * INCREMENTS);

    printf("Actual counter value: %d\n", counter);
}
```

Output:

```
screenikhilreddy@Strees-MacBook-Pro ~ % nano 09.c
screenikhilreddy@Strees-MacBook-Pro ~ % gcc 09.c -o 09
screenikhilreddy@Strees-MacBook-Pro ~ % ./09
Expected counter value: 400000
Actual counter value: 400000
screenikhilreddy@Strees-MacBook-Pro ~ %
```